

The Sage Programming Language

A Complete Guide to Systems Programming with Sage

SageLang Project

April 2026

Contents

1	Part I: Language Fundamentals	7
2	Introduction	8
2.1	Key Features	8
2.2	Quick Start	9
3	Variables and Types	10
3.1	Variable Declaration	10
3.2	Built-in Types	10
3.3	Type Checking	11
3.4	Type Conversion	11
4	Operators	12
4.1	Arithmetic Operators	12
4.2	Comparison Operators	12
4.3	Logical Operators	12
4.4	Bitwise Operators	13
4.5	String Concatenation	13
5	Control Flow	14
5.1	If / Elif / Else	14
5.2	While Loops	15
5.3	For Loops	15
5.4	Break and Continue	15
5.5	Match / Case	16
6	Functions	17
6.1	Basic Functions	17
6.2	Return Values	17
6.3	Default Parameters	17
6.4	Recursion	18
6.5	Closures	18
6.5.1	Stateful Closures	18
6.6	First-Class Functions	19
7	Strings	20
7.1	String Basics	20

7.2	Escape Sequences	20
7.3	String Functions	21
7.4	String Searching	21
7.5	Character Conversions	21
8	Data Structures	22
8.1	Arrays	22
8.2	Dictionaries	23
8.3	Tuples	23
9	Part II: Advanced Language Features	24
10	Object-Oriented Programming	25
10.1	Classes	25
10.2	Inheritance	25
10.3	Deep Inheritance	26
10.4	Arrow Operator	27
11	Structs, Enums, and Traits	28
11.1	Structs	28
11.2	Enums	28
11.3	Traits	29
12	Exception Handling	30
12.1	Basic Try/Catch	30
12.2	Try/Catch/Finally	30
12.3	Raising Exceptions	30
12.4	Nested Exception Handling	31
12.5	Custom Exception Objects	31
13	Generators and Iterators	32
13.1	Generator Functions	32
13.2	Yielding Values	32
14	Async and Concurrency	34
14.1	Async Procedures	34
14.2	Threads	34
14.3	Thread Sleep	35
14.4	True Atomic Operations	35
14.5	Semaphores	35
14.6	SMP and Multicore	35
15	Pattern Matching	37
15.1	Basic Matching	37
15.2	Guards	37
15.3	Matching Strings	38
16	Defer Statements	39

17 Module System	40
17.1 Importing Modules	40
17.2 Creating Modules	40
17.3 Standard Library Modules	41
18 Part III: Safety and Type System	42
19 Type Annotations	43
19.1 Parameter Types	43
19.2 Optional Types	43
19.3 Generic Type Parameters	44
20 The Safety System	45
20.1 Safety Modes	45
20.2 Ownership and Move Semantics	45
20.3 Borrow Checker	45
20.4 Lifetimes	46
20.5 Option Types	46
20.6 Fearless Concurrency: Send and Sync	46
20.7 Unsafe Blocks	47
20.8 Running the Safety Analyzer	47
21 Part IV: Standard Library	48
22 Built-in Modules	49
22.1 Math Module	49
22.2 IO Module	50
22.3 String Module	50
22.4 Sys Module	51
22.5 Thread Module	52
23 Collection Libraries	53
23.1 Arrays Module	53
23.2 Strings Module	53
23.3 Dicts Module	54
23.4 Iterator Module	54
23.5 Stats Module	54
23.6 Assert Module	55
24 JSON Module	56
25 Safety Library	57
26 Part V: Compilation and Backends	58
27 Compilation Backends	59
27.1 C Backend	59
27.2 LLVM Backend	59
27.3 Native Assembly Backend	59

27.4 Optimization Levels	60
27.5 Debug Info	60
28 Bare-Metal and OS Development	61
28.1 Supported Architectures	61
28.2 Quick Start: Build a Kernel with Sage	61
28.3 Example: x86_64 Kernel	62
28.4 Example: aarch64 Kernel	62
28.5 Example: RISC-V 64 Kernel	63
28.6 Boot Library Reference	64
28.7 Kernel Libraries	64
28.8 Debugging with GDB	65
29 Part VI: Tooling	66
30 Developer Tools	67
30.1 REPL	67
30.2 Formatter	67
30.3 Linter	67
30.4 Type Checker	68
30.5 Safety Analyzer	68
30.6 Language Server (LSP)	68
31 Build System	69
31.1 Makefile Targets	69
31.2 Version	69
32 Part VIb: Metaprogramming	70
33 Compile-Time Execution	71
33.1 Comptime Blocks	71
33.2 Comptime Expressions	71
34 Pragmas and Decorators	72
34.1 Syntax	72
34.2 Built-in Pragmas	72
34.3 Examples	72
34.4 Multiple Pragmas	73
35 AST Macros	74
35.1 Macro Definition	74
35.2 Future: Quote and Unquote	74
36 Generics	75
36.1 Generic Procedures	75
36.2 Generic Structs	75
36.3 Monomorphization (Planned)	75
37 Part VIc: Garbage Collection	76

37.1	GC Modes	76
37.1.1	Tracing GC (Default)	76
37.1.2	ARC (Automatic Reference Counting)	76
37.1.3	ORC (Optimized Reference Counting)	76
37.2	GC API	77
38	Part VIId: Kotlin/Android Backend	78
38.1	Emitting Kotlin	78
38.2	Building Android Apps	78
38.3	Type Specialization (-O2+)	79
38.4	Generators	79
38.5	Async/Await (Coroutines)	79
38.6	Memory Operations on Android	80
38.7	Jetpack Compose	80
38.8	GPU Graphics on Android	80
38.9	Concurrency on Android	80
38.10	Additional Mapped Builtins	81
39	Part VIe: Performance Optimization	82
39.1	Hybrid JIT/AOT Architecture	82
39.1.1	How It Works	82
39.1.2	Example	82
39.1.3	Profiling API	83
39.2	Performance Library (<code>lib/perf.sage</code>)	83
39.2.1	Signal Singletons	83
39.2.2	Dispatch Tables	83
39.2.3	Shape Objects	83
39.2.4	Fast Numeric Operations	83
39.2.5	Flat Environment Cache	84
39.3	Self-Hosted Interpreter Optimizations	84
39.4	Native C Interpreter Optimizations	84
39.5	Cross-Backend Benchmarks	84
40	Part VIId: Default Runtime and Execution Modes	86
40.1	JIT+AOT Hybrid Default	86
40.2	Runtime Pragma Decorators	86
41	Part VIg: SageMetal VM — Bare-Metal Bytecode Interpreter	88
41.1	Architecture	88
41.2	MetalValue Type	88
41.3	Host I/O Callbacks	88
41.4	Single-Step Mode	89
41.5	Building	89
42	Part VIh: Metal Standard Library	90
42.1	<code>metal.core</code>	90
42.2	<code>metal.serial</code>	90
42.3	<code>metal.irq</code>	91

42.4	metal.timer	91
42.5	metal.gpio	91
43	Part VII: Appendices	93
44	Appendix A: Built-in Functions	94
44.1	Core Functions	94
44.2	Array Functions	94
44.3	String Functions	95
44.4	Dictionary Functions	95
44.5	Generator Functions	95
44.6	GC Functions	95
44.7	Path Functions	96
44.8	Bytes Functions	96
44.9	Memory Functions	96
44.10	FFI Functions	96
44.11	Struct Interop Functions	97
44.12	Assembly Functions	97
45	Appendix B: CLI Reference	98
46	Appendix C: Language Gotchas	100
46.1	Still Relevant	100
46.2	Fixed in v1.4+	100
47	Appendix D: Reserved Keywords	102
47.1	Control Flow	102
47.2	Functions and Classes	102
47.3	Variables	103
47.4	Values and Logic	103
47.5	Exceptions	103
47.6	Advanced	103

Chapter 1

Part I: Language Fundamentals

Chapter 2

Introduction

Sage is a clean, indentation-based systems programming language built in C. It combines the readability of Python with the performance of compiled languages, offering multiple compilation backends, a compile-time safety system inspired by Rust, and a self-hosted compiler written in Sage itself.

2.1 Key Features

- **Python-like syntax** with indentation-based blocks and explicit **end** keywords
- **9 execution backends**: C codegen, LLVM IR, native x86-64/aarch64/rv64 assembly, bytecode VM, JIT, AOT, Kotlin/Android
- **Compile-time safety**: ownership tracking, borrow checker, lifetimes, Option types
- **Self-hosted compiler** written in Sage (lexer, parser, interpreter, codegen) with dispatchable optimizations
- **Rich standard library**: 23+ modules including math, io, string, sys, thread, JSON, collections
- **Object-oriented programming** with classes, inheritance, structs, enums, and traits
- **Generators and async**: yield-based generators, async/await concurrency
- **Three GC modes**: tracing (concurrent mark-sweep), ARC (reference counting), ORC (optimized RC with cycle detection)
- **Kotlin/Android transpiler**: `--emit-kotlin` and `--compile-android` generate complete Gradle projects from a single `.sage` file
- **Developer tooling**: formatter, linter, type checker, LSP server, REPL
- **Bare-metal and OS development**: Multiboot2, UEFI, QEMU integration
- **GPU programming**: Vulkan and OpenGL backends (covered in separate guides)
- **Machine learning**: neural networks, model training (covered in separate guides)
- **Performance library** (`lib/perf.sage`): dispatch tables, signal singletons, flat env cache, shape constructors
- **SageMetal VM**: freestanding bytecode interpreter for bare-metal (no malloc, no libc, no OS)
- **Metal stdlib** (`lib/metal/`): serial, GPIO, IRQ, timer, MMIO for kernel/embedded development
- **Default hybrid runtime**: JIT profiling on hosted, AST on bare-metal, automatic selection
- **304+ interpreter tests**, 1987+ self-hosted tests

2.2 Quick Start

Build from source

```
git clone https://github.com/Night-Traders-Dev/SageLang.git
```

```
cd SageLang
```

```
make
```

Run a program

```
./sage hello.sage
```

Interactive REPL

```
./sage
```

A minimal Sage program:

```
print "Hello, World!"
```

A slightly larger program demonstrating functions and control flow:

```
proc fizzbuzz(n):  
  for i in range(1, n + 1):  
    if i % 15 == 0:  
      print "FizzBuzz"  
    elif i % 3 == 0:  
      print "Fizz"  
    elif i % 5 == 0:  
      print "Buzz"  
    else:  
      print i  
  end  
end  
end  
  
fizzbuzz(20)
```

Chapter 3

Variables and Types

Sage supports `let` for variable declaration and `var` for mutable variables.

3.1 Variable Declaration

```
# Immutable binding
let name = "Alice"
let age = 30
let pi = 3.14159
let active = true
let nothing = nil

# Mutable variable
var counter = 0
counter = counter + 1
print counter      # 1
```

Variables declared with `let` cannot be reassigned. Use `var` when you need a variable whose value changes over time.

3.2 Built-in Types

Type	Example	Description
Number	42, 3.14, 0xFF, 0o755	Integer or floating-point
String	"hello"	Text with escape sequences
Boolean	true, false	Logical values
Nil	nil	Absence of value
Array	[1, 2, 3]	Ordered, mutable collection
Dict	{"a": 1}	Key-value mapping
Tuple	(1, "x")	Immutable ordered pair

Sage numbers are IEEE 754 double-precision floating point. Integer values are exact up to 2^{53} . Hex literals (0xFF) and octal literals (0o755) are supported as of v1.4.

3.3 Type Checking

The `type()` function returns the type of any value as a string:

```
print type(42)           # "number"
print type("hello")      # "string"
print type(true)         # "bool"
print type(nil)          # "nil"
print type([1, 2])       # "array"
print type({"a": 1})      # "dict"
print type((1, 2))       # "tuple"
```

3.4 Type Conversion

```
# Number to string
print str(42)            # "42"
print str(3.14)          # "3.14"
print str(true)          # "true"
print str(nil)           # "nil"

# String to number
print tonumber("123")    # 123
print tonumber("3.14")   # 3.14

# Truncate to integer
print int(3.7)           # 3
print int(-2.9)          # -2

# Character conversions
print chr(65)            # "A"
print ord("A")           # 65
print chr(10)            # newline character
```

Chapter 4

Operators

4.1 Arithmetic Operators

```
print 5 + 2      # 7   (addition)
print 5 - 2      # 3   (subtraction)
print 5 * 2      # 10  (multiplication)
print 5 / 2      # 2.5 (division)
print 5 % 2      # 1   (modulo, uses fmod for floats)
print -5         # -5  (negation)
```

Operator precedence follows standard math rules

```
print 2 + 3 * 4      # 14 (not 20)
print (2 + 3) * 4     # 20
```

The % operator uses `fmod()` internally, so it preserves float semantics:

```
print 7.5 % 2.5      # 0
print 3.7 % 1.0      # 0.7
```

4.2 Comparison Operators

```
print 1 == 1        # true
print 1 != 2        # true
print 1 < 2          # true
print 2 > 1          # true
print 1 <= 1         # true
print 1 >= 1         # true
```

Comparison works structurally for arrays, dicts, and class instances. Two instances with the same fields and values compare as equal.

4.3 Logical Operators

```
print true and false # false
print true or false  # true
```

```
print not true          # false
```

Important: In Sage, 0 is **truthy**. Only **false** and **nil** are falsy. Use explicit comparisons: `if x == 0:` rather than `if not x:`.

4.4 Bitwise Operators

```
print 5 & 3      # 1  (AND)
print 5 | 3      # 7  (OR)
print 5 ^ 3      # 6  (XOR)
print ~5        # -6 (NOT)
print 1 << 4     # 16 (left shift)
print 16 >> 2    # 4  (right shift)
```

4.5 String Concatenation

Strings are concatenated with `+`:

```
let greeting = "Hello" + ", " + "World!"
print greeting      # Hello, World!

# Convert other types before concatenation
let msg = "Count: " + str(42)
print msg           # Count: 42
```

Chapter 5

Control Flow

5.1 If / Elif / Else

Conditional blocks use `if`, `elif`, `else`, and are terminated with `end`:

```
let x = 42

if x > 100:
    print "big"
elif x > 10:
    print "medium"
else:
    print "small"
end
```

The `elif` keyword supports unlimited branches. Sage processes them as chained if/else structures internally.

```
let grade = 85

if grade >= 90:
    print "A"
elif grade >= 80:
    print "B"
elif grade >= 70:
    print "C"
elif grade >= 60:
    print "D"
else:
    print "F"
end
```

5.2 While Loops

```
var i = 0
while i < 5:
    print i
    i = i + 1
end
```

5.3 For Loops

```
# Iterate over arrays
let fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print fruit
end
```

```
# Range-based for loop
for i in range(5):
    print i      # 0, 1, 2, 3, 4
end
```

```
# Range with start and end
for i in range(2, 5):
    print i      # 2, 3, 4
end
```

5.4 Break and Continue

```
# Break exits the loop
var i = 0
while true:
    if i == 5:
        break
    end
    print i
    i = i + 1
end
```

```
# Continue skips to the next iteration
for x in range(10):
    if x % 2 == 0:
        continue
    end
    print x      # 1, 3, 5, 7, 9
end
```

5.5 Match / Case

Pattern matching with optional guards:

```
let color = "red"
match color:
  case "red":
    print "Stop"
  end
  case "yellow":
    print "Caution"
  end
  case "green":
    print "Go"
  end
  default:
    print "Unknown color"
  end
end
```

Match with guards using if:

```
let score = 85
match score:
  case score if score >= 90:
    print "Excellent"
  end
  case score if score >= 70:
    print "Good"
  end
  default:
    print "Needs improvement"
  end
end
```

Chapter 6

Functions

Functions are declared with `proc` and terminated with `end`.

6.1 Basic Functions

```
proc greet(name):  
  print "Hello, " + name + "!"  
end  
  
greet("Alice")    # Hello, Alice!
```

6.2 Return Values

```
proc add(a, b):  
  return a + b  
end  
  
let result = add(3, 4)  
print result    # 7
```

6.3 Default Parameters

```
proc connect(host, port, ssl):  
  print "Connecting to " + host + ":" + str(port)  
end  
  
proc greet_user(name, greeting):  
  print greeting + ", " + name + "!"  
end  
  
greet_user("Alice", "Hello")    # Hello, Alice!  
greet_user("Bob", "Hi")         # Hi, Bob!
```

6.4 Recursion

```
proc factorial(n):  
  if n <= 1:  
    return 1  
  end  
  return n * factorial(n - 1)  
end  
  
print factorial(5)      # 120  
  
proc fibonacci(n):  
  if n <= 1:  
    return n  
  end  
  return fibonacci(n - 1) + fibonacci(n - 2)  
end  
  
print fibonacci(10)     # 55
```

6.5 Closures

Functions capture their lexical scope, enabling closures:

```
proc make_adder(n):  
  proc add(x):  
    return x + n  
  end  
  return add  
end  
  
let add5 = make_adder(5)  
print add5(10)      # 15  
print add5(20)      # 25
```

6.5.1 Stateful Closures

```
proc make_counter():  
  var count = 0  
  proc increment():  
    count = count + 1  
    return count  
  end  
  return increment  
end  
  
let counter = make_counter()  
print counter()      # 1
```

```
print counter()    # 2
print counter()    # 3
```

6.6 First-Class Functions

Functions are values and can be passed as arguments:

```
proc apply(f, x):
  return f(x)
end

proc double(n):
  return n * 2
end

print apply(double, 5)    # 10

# Anonymous-style via closures
let square = proc(x):
  return x * x
end

print square(4)    # 16
```

Chapter 7

Strings

Sage strings support escape sequences and a rich set of built-in operations.

7.1 String Basics

```
let greeting = "Hello, World!"
print len(greeting)    # 13

# Character access by index
print greeting[0]      # "H"
print greeting[7]      # "W"
```

7.2 Escape Sequences

As of v1.4, Sage supports the following escape sequences in string literals:

Escape	Character
\n	Newline (LF)
\t	Tab
\r	Carriage return
\\	Backslash
\"	Double quote
\'	Single quote
\0	Null byte
\a	Bell
\b	Backspace
\f	Form feed
\v	Vertical tab
\xHH	Hex byte (e.g. \x41 = “A”)

```
print "Line one\nLine two"
print "Column1\tColumn2"
```

```
print "She said \\\"hi\\\""
print "Hex: \x48\x65\x6C\x6C\x6F"      # "Hello"
```

7.3 String Functions

```
# Case conversion
print upper("hello")                    # "HELLO"
print lower("HELLO")                    # "hello"

# Trimming
print strip("  hello  ")                # "hello"

# Replacement
print replace("hello", "l", "r")        # "herro"

# Splitting and joining
let parts = split("a,b,c", ",")
print parts[0]                          # "a"
print join(parts, "-")                   # "a-b-c"
```

7.4 String Searching

```
print startswith("hello", "hel")        # true
print endswith("hello", "llo")          # true
print contains("hello world", "world")  # true
print indexof("hello", "ll")            # 2
```

7.5 Character Conversions

```
print chr(65)      # "A"
print chr(10)      # newline
print ord("A")     # 65
print ord("z")     # 122
```

Chapter 8

Data Structures

8.1 Arrays

Arrays are ordered, mutable collections:

```
let nums = [1, 2, 3, 4, 5]
print nums[0]          # 1
print len(nums)        # 5

# Push and pop
push(nums, 6)
print len(nums)        # 6
let last = pop(nums)
print last              # 6

# Both push() and append() work identically
append(nums, 7)
print len(nums)        # 6

# Slicing
let sub = slice(nums, 1, 3)
print sub               # [2, 3]

# Nested arrays
let matrix = [[1, 2], [3, 4], [5, 6]]
print matrix[1][0]      # 3

# Iteration
var total = 0
for n in nums:
    total = total + n
end
print total
```

8.2 Dictionaries

Dictionaries are key-value mappings with string keys:

```
let person = {"name": "Alice", "age": 30}
print person["name"]      # "Alice"

# Assignment
person["email"] = "alice@example.com"

# Check keys
print dict_has(person, "name")    # true
print dict_has(person, "phone")   # false

# Get keys and values
let keys = dict_keys(person)
let vals = dict_values(person)

# Delete a key
dict_delete(person, "age")

# Build incrementally (no multiline dict literals)
let config = {}
config["host"] = "localhost"
config["port"] = 8080
config["ssl"] = true
```

8.3 Tuples

Tuples are immutable ordered collections:

```
let point = (3, 7)
print point[0]    # 3
print point[1]    # 7
print len(point)  # 2
```

Chapter 9

Part II: Advanced Language Features

Chapter 10

Object-Oriented Programming

10.1 Classes

Classes are defined with `class` and terminated with `end`. The `init` method serves as the constructor, and `self` refers to the current instance:

```
class Person:
  proc init(self, name, age):
    self.name = name
    self.age = age
  end

  proc greet(self):
    return "Hi, I'm " + self.name
  end
end

let p = Person("Alice", 30)
print p.name      # Alice
print p.greet()   # Hi, I'm Alice
```

10.2 Inheritance

Classes can inherit from a parent class. Use `super` to call parent methods:

```
class Animal:
  proc init(self, name):
    self.name = name
  end

  proc speak(self):
    return "..."
  end
end
```

```
class Dog(Animal):
  proc init(self, name, breed):
    super.init(self, name)
    self.breed = breed
  end

  proc speak(self):
    return "Woof!"
  end
end

let d = Dog("Rex", "German Shepherd")
print d.name      # Rex
print d.breed     # German Shepherd
print d.speak()   # Woof!
```

Note that `super` requires explicit `self` as the first argument: `super.init(self, args)`.

10.3 Deep Inheritance

```
class A:
  proc init(self, x):
    self.x = x
  end
end

class B(A):
  proc init(self, x, y):
    super.init(self, x)
    self.y = y
  end
end

class C(B):
  proc init(self, x, y, z):
    super.init(self, x, y)
    self.z = z
  end
end

let obj = C(1, 2, 3)
print obj.x    # 1
print obj.y    # 2
print obj.z    # 3
```

10.4 Arrow Operator

The `->` operator is an alias for `.`, allowing C-style member access:

```
class Vec3:
  proc init(self, x, y, z):
    self->x = x
    self->y = y
    self->z = z
  end

  proc mag_squared(self):
    return self->x * self->x + self->y * self->y + self->z * self->z
  end
end

let v = Vec3(3, 4, 0)
print v->x           # 3
print v->mag_squared() # 25
```

Chapter 11

Structs, Enums, and Traits

11.1 Structs

Structs define data types with typed fields:

```
struct Point:  
  x: Int  
  y: Int  
end
```

```
struct Color:  
  r: Int  
  g: Int  
  b: Int  
end
```

Struct definitions create constructor-like classes. Fields are accessed with dot notation.

11.2 Enums

Enums define a fixed set of named variants:

```
enum Direction:  
  North  
  South  
  East  
  West  
end
```

```
enum Color:  
  Red  
  Green  
  Blue  
end
```

Each variant is accessible as `Direction.North`, `Color.Red`, etc.

11.3 Traits

Traits define method signatures that classes can implement:

```
trait Printable:
  proc to_string(self) -> String
end

trait Comparable:
  proc compare(self, other) -> Int
end
```

Traits provide a form of interface specification. Classes implementing a trait should provide all the methods declared in the trait.

Chapter 12

Exception Handling

Exception handling uses `try`, `catch`, `finally`, and `raise`, with blocks terminated by `end`:

12.1 Basic Try/Catch

```
try:
  let result = 10 / 0
catch e:
  print "Error: " + e
end
```

12.2 Try/Catch/Finally

The `finally` block always executes, whether or not an exception was raised:

```
try:
  print "Opening resource"
  raise "Something went wrong"
catch e:
  print "Caught: " + e
finally:
  print "Cleanup complete"
end
```

12.3 Raising Exceptions

```
proc divide(a, b):
  if b == 0:
    raise "Division by zero"
  end
  return a / b
end
```

```
try:
    print divide(10, 0)
catch e:
    print e      # Division by zero
end
```

12.4 Nested Exception Handling

```
try:
    try:
        raise "inner error"
    catch e:
        print "Inner: " + e
    end
    print "Outer continues"
catch e:
    print "Should not reach here"
end
```

12.5 Custom Exception Objects

You can raise any value as an exception, including dicts for structured errors:

```
proc validate_age(age):
    if age < 0:
        let err = {}
        err["type"] = "ValidationError"
        err["message"] = "Age cannot be negative"
        err["value"] = age
        raise err
    end
    return age
end

try:
    validate_age(-5)
catch e:
    print "Error: " + str(e)
end
```

Chapter 13

Generators and Iterators

13.1 Generator Functions

A function containing `yield` becomes a generator. Use `next()` to advance it:

```
proc count_up(start, limit):  
  var i = start  
  while i < limit:  
    yield i  
    i = i + 1  
  end  
end
```

```
let gen = count_up(0, 5)  
print next(gen)    # 0  
print next(gen)    # 1  
print next(gen)    # 2  
print next(gen)    # 3  
print next(gen)    # 4
```

13.2 Yielding Values

`yield` suspends the function and returns a value to the caller. The function resumes from the point after `yield` when `next()` is called again:

```
proc fibonacci_gen():  
  var a = 0  
  var b = 1  
  while true:  
    yield a  
    let temp = a  
    a = b  
    b = temp + b  
  end
```

```
end

let fib = fibonacci_gen()
for i in range(10):
    print next(fib)
end
# Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Chapter 14

Async and Concurrency

14.1 Async Procedures

Sage supports `async proc` for concurrent execution. Use `await` to wait for the result:

```
async proc fetch_data(url):  
    # Simulated async work  
    return "data from " + url  
end  
  
let result = await fetch_data("https://example.com")  
print result
```

14.2 Threads

The `thread` module provides OS-level threading with mutexes:

```
import thread  
  
var shared = 0  
let mtx = thread.mutex()  
  
proc worker(id):  
    for i in range(1000):  
        thread.lock(mtx)  
        shared = shared + 1  
        thread.unlock(mtx)  
    end  
end  
  
let t1 = thread.spawn(worker, 1)  
let t2 = thread.spawn(worker, 2)  
thread.join(t1)  
thread.join(t2)
```

```
print shared      # 2000
```

14.3 Thread Sleep

```
import thread

print "Starting..."
thread.sleep(1.5)    # Sleep for 1.5 seconds
print "Done!"
```

14.4 True Atomic Operations

Sage provides C-level atomic operations via `__atomic` compiler builtins. These are truly atomic — safe for concurrent access from multiple threads without locks.

```
let counter = atomic_new(0)      # Create atomic integer, initial value 0

# These are safe to call from multiple threads simultaneously:
atomic_add(counter, 1)           # Atomic increment
atomic_add(counter, 5)           # Atomic add
let val = atomic_load(counter)   # Atomic read (returns 6)

# Compare-and-swap (CAS):
let ok = atomic_cas(counter, 6, 10) # If counter == 6, set to 10
# ok == true, counter is now 10

# Atomic exchange:
let old = atomic_exchange(counter, 0) # Set to 0, return old value (10)
```

14.5 Semaphores

POSIX semaphores for controlling access to a finite number of resources:

```
let sem = sem_new(3)            # 3 permits available

sem_wait(sem)                   # Acquire permit (blocks if none available)
sem_wait(sem)                   # Acquire another
# ... do work with the resource ...
sem_post(sem)                   # Release permit

let ok = sem_trywait(sem)       # Non-blocking acquire (returns true/false)
```

14.6 SMP and Multicore

CPU topology detection and core affinity:

```
# Detect hardware
let cores = cpu_count()           # Logical CPUs (includes hyperthreads)
let physical = cpu_physical_cores() # Physical cores only
let ht = cpu_has_hyperthreading() # true if SMT/HT detected

# Pin thread to specific core
thread_set_affinity(0)           # Pin to core 0
let core = thread_get_core() # Which core am I on?

Multicore work distribution using lib/os/smp.sage:

import os.smp

smp.print_topology() # Print CPU info

# Run work on all cores in parallel
let results = smp.on_all_cores(proc(core_id):
    return core_id * core_id
)

# Distribute array work across cores
let items = range(10000)
let sums = smp.parallel_for_cores(items, proc(core_id, slice):
    let total = 0
    for x in slice:
        total = total + x
    return total
)
```

Chapter 15

Pattern Matching

The `match` statement provides pattern matching with guards.

15.1 Basic Matching

```
let status = 404

match status:
  case 200:
    print "OK"
  end
  case 301:
    print "Moved"
  end
  case 404:
    print "Not Found"
  end
  case 500:
    print "Server Error"
  end
  default:
    print "Unknown status: " + str(status)
  end
end
```

15.2 Guards

Add conditions to cases with `if`:

```
let value = 42

match value:
  case value if value < 0:
    print "Negative"
```

```
end
case value if value == 0:
    print "Zero"
end
case value if value > 0 and value < 100:
    print "Small positive"
end
default:
    print "Large"
end
end
```

15.3 Matching Strings

```
proc http_method(method):
    match method:
        case "GET":
            return "Retrieving resource"
        end
        case "POST":
            return "Creating resource"
        end
        case "PUT":
            return "Updating resource"
        end
        case "DELETE":
            return "Deleting resource"
        end
        default:
            return "Unknown method"
        end
    end
end

print http_method("GET")    # Retrieving resource
```

Chapter 16

Defer Statements

The `defer` statement schedules a statement to execute when the current scope exits, regardless of how it exits (normal return or exception):

```
proc process_file(path):  
  let data = "opened"  
  defer print "Closing file"  
  
  print "Processing: " + path  
  # ... do work ...  
  return data  
end
```

```
process_file("test.txt")  
# Output:  
#   Processing: test.txt  
#   Closing file
```

Deferred statements execute in LIFO (last-in, first-out) order:

```
proc example():  
  defer print "First deferred"  
  defer print "Second deferred"  
  defer print "Third deferred"  
  print "Main body"  
end
```

```
example()  
# Output:  
#   Main body  
#   Third deferred  
#   Second deferred  
#   First deferred
```

Chapter 17

Module System

17.1 Importing Modules

```
# Import entire module
import math
print math.abs(-42)      # 42
print math.sqrt(16)     # 4

# Import specific items
from strings import pad_left
print pad_left("42", 5, "0")  # "00042"

# Import with alias
import arrays as arr
let nums = [1, 2, 3]

# Import with item alias
from math import sqrt as square_root
print square_root(25)      # 5
```

17.2 Creating Modules

Any `.sage` file is a module. Top-level definitions are exported automatically:

```
# mylib.sage
let VERSION = "1.0"

proc hello(name):
    return "Hello, " + name
end

proc add(a, b):
    return a + b
end
```

```
# main.sage
import mylib
print mylib.VERSION          # 1.0
print mylib.hello("World")  # Hello, World
print mylib.add(2, 3)       # 5
```

17.3 Standard Library Modules

The following native modules are built into the interpreter:

Module	Description
math	Mathematical functions and constants
io	File I/O operations
string	String searching, manipulation
sys	System info, args, env, exit
thread	OS-level threading and mutexes

The following bundled Sage modules are in the `lib/` directory:

Module	Description
arrays	Functional array operations
strings	String formatting and transformation
dicts	Dictionary utilities
iter	Iterator combinators and generators
stats	Statistical functions
utils	General-purpose utilities
assert	Testing assertions
math	Extended math (Sage-level supplement)
json	Full JSON parser and serializer

Chapter 18

Part III: Safety and Type System

Chapter 19

Type Annotations

Sage supports optional type annotations on function parameters and return types.

19.1 Parameter Types

```
proc add(a: Int, b: Int) -> Int:
  return a + b
end

proc greet(name: String) -> String:
  return "Hello, " + name
end

proc is_even(n: Int) -> Bool:
  return n % 2 == 0
end
```

19.2 Optional Types

The T? syntax marks a type as optional (may be nil):

```
proc find_user(id: Int) -> String?:
  if id == 1:
    return "Alice"
  end
  return nil
end

let user = find_user(99)
if user != nil:
  print user
end
```

19.3 Generic Type Parameters

Type annotations support generic parameters for collections:

```
proc sum_list(nums: Array[Int]) -> Int:
  var total = 0
  for n in nums:
    total = total + n
  end
  return total
end
```

Type annotations are checked by the type checker (`sage check`) and the safety system but are not enforced at runtime by the interpreter.

Chapter 20

The Safety System

Sage v3.1.3 includes a compile-time safety system inspired by Rust. It provides ownership tracking, borrow checking, lifetime analysis, Option type enforcement, and fearless concurrency checks.

The safety system is a **static analysis pass** that runs after parsing and before code generation. It does not affect the interpreter.

20.1 Safety Modes

Mode	Flag	Description
Off	(default)	No safety checks
Annotated	<code>--safety</code>	Only check <code>@safe</code> functions
Strict	<code>--strict-safety</code>	Enforce everything globally

20.2 Ownership and Move Semantics

Variables own their data. When a value is assigned to another variable, ownership is transferred (moved), and the original variable becomes invalid:

```
# In strict safety mode:
let data = [1, 2, 3]
let other = data      # ownership moves to 'other'
# print data          # ERROR: use after move
print other           # OK: [1, 2, 3]
```

20.3 Borrow Checker

The borrow checker enforces two rules:

1. You can have **any number of immutable borrows** at the same time.
2. You can have **exactly one mutable borrow** at a time.
3. You cannot have both mutable and immutable borrows simultaneously.

```
# Immutable borrows are fine together
let x = 42
let a = x    # immutable borrow
let b = x    # immutable borrow -- OK

# Mutable borrow is exclusive
var y = 10
# Only one mutable reference at a time
```

20.4 Lifetimes

Lifetimes ensure references do not outlive the data they point to:

```
proc get_ref():
  let local = "hello"
  return local    # WARNING: reference may outlive local data
end
```

20.5 Option Types

In safe contexts, nil values must be handled explicitly using Option semantics:

```
proc find(items, target):
  for item in items:
    if item == target:
      return item    # Some(item)
    end
  end
  return nil        # None
end

let result = find([1, 2, 3], 2)
# In strict mode, must check before use:
if result != nil:
  print result
end
```

The safety library provides `Some()`, `None()`, `unwrap()`, and `map()` for working with Option types explicitly.

20.6 Fearless Concurrency: Send and Sync

The safety system tracks `Send` and `Sync` traits:

- **Send:** A type is `Send` if ownership can be transferred between threads.
- **Sync:** A type is `Sync` if it can be safely shared between threads.

In strict mode, passing a non-`Send` value to `thread.spawn` is an error.

20.7 Unsafe Blocks

Operations that bypass safety checks must be wrapped in `unsafe`:

```
unsafe:
  let ptr = mem_alloc(1024)
  mem_write(ptr, 0, 42)
  let val = mem_read(ptr, 0)
  mem_free(ptr)
end
```

20.8 Running the Safety Analyzer

```
# Analyze with default (annotated) mode
./sage --safety program.sage
```

```
# Analyze with strict mode
./sage --strict-safety program.sage
```

```
# Via the sage command
./sage safety program.sage
```

Chapter 21

Part IV: Standard Library

Chapter 22

Built-in Modules

22.1 Math Module

The `math` module provides trigonometric, logarithmic, and utility functions:

```
import math

# Trigonometry
print math.sin(math.pi / 2)    # 1.0
print math.cos(0)               # 1.0
print math.tan(math.pi / 4)    # ~1.0
print math.asin(1.0)           # ~1.5708
print math.acos(0.0)           # ~1.5708
print math.atan(1.0)           # ~0.7854
print math.atan2(1.0, 1.0)     # ~0.7854

# Powers and roots
print math.sqrt(16)            # 4
print math.pow(2, 10)          # 1024
print math.log(math.e)         # 1
print math.log10(1000)         # 3
print math.exp(1)              # ~2.71828

# Rounding
print math.floor(3.7)          # 3
print math.ceil(3.2)           # 4
print math.round(3.5)          # 4
print math.abs(-42)            # 42
print math.fmod(7, 3)          # 1

# Min/max/clamp
print math.min(3, 7)           # 3
print math.max(3, 7)           # 7
print math.clamp(150, 0, 100) # 100
```

```
# Checks
print math.isnan(0.0 / 0.0)    # true
print math.isinf(1.0 / 0.0)    # true

# Constants
print math.pi                 # 3.14159265358979
print math.e                   # 2.71828182845905
print math.tau                 # 6.28318530717959
print math.inf                 # Infinity
print math.nan                 # NaN

# Random
print math.random()           # random float in [0, 1)
```

22.2 IO Module

The `io` module provides file system operations:

```
import io

# Read and write files
let content = io.readfile("data.txt")
io.writefile("output.txt", "Hello, World!")
io.appendfile("log.txt", "New entry\n")

# File system queries
print io.exists("data.txt")      # true/false
print io.isdir("/home")         # true
print io.filesize("data.txt")    # file size in bytes

# Binary I/O
let bytes = io.readbytes("image.png")
print len(bytes)                # number of bytes

# Directory listing
let files = io.listdir(".")
for f in files:
  print f
end

# Remove files
io.remove("temp.txt")
```

22.3 String Module

The `string` module provides advanced string operations:

```

import string

# Search
print string.find("hello world", "world")      # 6
print string.rfind("hello hello", "hello")     # 6
print string.startswith("hello", "hel")        # true
print string.endswith("hello", "llo")          # true
print string.contains("hello world", "world")   # true

# Character access
print string.char_at("hello", 1)                # "e"
print string.ord("A")                          # 65
print string.chr(65)                          # "A"

# Manipulation
print string.repeat("ab", 3)                    # "ababab"
print string.count("banana", "an")              # 2
print string.substr("hello", 1, 3)              # "ell"
print string.reverse("hello")                   # "olleh"

```

22.4 Sys Module

The `sys` module provides system-level utilities:

```

import sys

# Command-line arguments
let args = sys.args()
for arg in args:
  print arg
end

# Environment
print sys.platform      # "linux", "darwin", or "windows"
print sys.version       # "2.1.0"
print sys.getenv("HOME") # /home/user

# Timing
let start = sys.clock()
# ... do work ...
let elapsed = sys.clock() - start
print "Took " + str(elapsed) + " seconds"

# Sleep
sys.sleep(0.5)          # sleep for 0.5 seconds

# Exit
sys.exit(0)

```

22.5 Thread Module

The `thread` module provides OS-level concurrency:

```
import thread

# Spawn a thread
proc worker(id):
    print "Worker " + str(id) + " running"
end

let t = thread.spawn(worker, 1)
thread.join(t)

# Mutexes for synchronization
let mtx = thread.mutex()
thread.lock(mtx)
# ... critical section ...
thread.unlock(mtx)

# Sleep
thread.sleep(0.1)    # sleep for 100ms
```

Chapter 23

Collection Libraries

23.1 Arrays Module

```
import arrays

let nums = [5, 3, 8, 1, 9, 2]

# Functional operations
let doubled = arrays.map(nums, proc(x): return x * 2 end)
let evens = arrays.filter(nums, proc(x): return x % 2 == 0 end)
let total = arrays.reduce(nums, 0, proc(acc, x): return acc + x end)

# Search
print arrays.contains(nums, 8)      # true
print arrays.index_of(nums, 8)     # 2

# Transform
let rev = arrays.reverse(nums)
let combined = arrays.concat([1, 2], [3, 4])
```

23.2 Strings Module

```
import strings

let text = " Hello, World! "
print strings.compact(text)          # "Hello, World!"
print strings.pad_left("42", 5, "0") # "00042"
print strings.pad_right("hi", 10, ".") # "hi....."
print strings.repeat("ab", 3)        # "ababab"
print strings.surround("hello", "[", "]") # "[hello]"
print strings.snake_case("HelloWorld") # "hello_world"
print strings.dash_case("helloWorld")  # "hello-world"
print strings.csv(["a", "b", "c"])     # "a,b,c"
```

23.3 Dicts Module

```
import dicts

let config = {"host": "localhost", "port": 8080, "debug": false}

print dicts.size(config)           # 3
print dicts.get_or(config, "timeout", 30) # 30 (default)
print dicts.has_all(config, ["host", "port"]) # true

let entries = dicts.entries(config)
for e in entries:
  print str(e[0]) + " = " + str(e[1])
end
```

23.4 Iterator Module

```
import iter

# Infinite counter
for n in iter.count(0, 2):
  if n > 10:
    break
  end
  print n      # 0, 2, 4, 6, 8, 10
end

# Enumeration
let fruits = ["apple", "banana", "cherry"]
for pair in iter.enumerate_array(fruits):
  print str(pair[0]) + ": " + pair[1]
end

# Take first N
let first5 = iter.take(iter.count(0, 1), 5)
```

23.5 Stats Module

```
import stats

let scores = [85, 92, 78, 95, 88, 91, 76]

print stats.mean(scores)      # ~86.4
print stats.min_value(scores) # 76
print stats.max_value(scores) # 95
print stats.range_span(scores) # 19
print stats.sum(scores)       # 605
```

23.6 Assert Module

```
import assert

let result = 2 + 2
assert.assert_equal(result, 4, "Basic addition")
assert.assert_true(result > 0, "Positive result")
assert.assert_not_nil(result, "Not nil")
assert.assert_close(3.14159, 3.14, 0.01, "Pi approximation")
```

Chapter 24

JSON Module

The `json` module is a full JSON parser and serializer written in pure Sage, ported from `cJSON`:

```
import json

# Parse a JSON string
let text = "{\"name\": \"Alice\", \"age\": 30}"
let data = json.cJSON_Parse(text)

# Access values
let name_item = json.cJSON_GetObjectItem(data, "name")
print json.cJSON_GetStringValue(name_item)    # Alice

let age_item = json.cJSON_GetObjectItem(data, "age")
print json.cJSON_GetNumberValue(age_item)    # 30

# Create JSON
let obj = json.cJSON_CreateObject()
json.cJSON_AddStringToObject(obj, "greeting", "hello")
json.cJSON_AddNumberToObject(obj, "count", 42)
json.cJSON_AddBoolToObject(obj, "active", true)

# Serialize to string
let output = json.cJSON_Print(obj)
print output

# Clean up
json.cJSON_Delete(data)
json.cJSON_Delete(obj)
```

The JSON module passes 88 tests and supports all JSON types: objects, arrays, strings, numbers, booleans, and null.

Chapter 25

Safety Library

The safety library provides explicit Option type operations for working with potentially-nil values:

```
# Option type constructors
let some_val = Some(42)
let no_val = None()

# Unwrap (panics if None)
let x = unwrap(some_val)    # 42

# Safe access with map
let doubled = map(some_val, proc(v): return v * 2 end)
# doubled is Some(84)

# Check before use
if some_val != nil:
  print unwrap(some_val)
end
```

When `--strict-safety` is enabled, the compiler enforces that all Option-typed values are checked before use, preventing nil-related runtime errors.

Chapter 26

Part V: Compilation and Backends

Chapter 27

Compilation Backends

Sage provides three compilation backends, each targeting different use cases.

27.1 C Backend

The C backend translates Sage to portable C code:

```
# Emit C source code
./sage --emit-c program.sage -o output.c

# Compile to binary via C (uses gcc/cc)
./sage --compile program.sage -o program

# Run the compiled binary
./program
```

The C backend is the most mature and supports the full language. The generated C code is portable and can be compiled with any C11-compliant compiler.

27.2 LLVM Backend

The LLVM backend generates LLVM IR text, then uses `clang` to compile:

```
# Emit LLVM IR
./sage --emit-llvm program.sage -o output.ll

# Compile to binary via LLVM
./sage --compile-llvm program.sage -o program
```

The LLVM backend supports classes, methods, exception handling, bitwise operators, and GPU operations. It links against `llvm_runtime.c` which provides 40+ runtime functions.

27.3 Native Assembly Backend

The native backend generates assembly directly for three architectures:

```

# Emit x86-64 assembly
./sage --emit-asm program.sage -o output.s --target x86-64

# Emit aarch64 assembly
./sage --emit-asm program.sage -o output.s --target aarch64

# Emit RISC-V 64-bit assembly
./sage --emit-asm program.sage -o output.s --target rv64

# Compile to native binary
./sage --compile-native program.sage -o program --target x86-64

```

27.4 Optimization Levels

All backends support optimization passes:

```

./sage -00 program.sage    # No optimization
./sage -01 program.sage    # Basic: constant folding
./sage -02 program.sage    # Standard: + dead code elimination
./sage -03 program.sage    # Aggressive: + function inlining

```

The optimization passes operate on the AST before code generation:

Level	Passes
-00	None
-01	Constant folding
-02	Constant folding, dead code elimination
-03	Constant folding, dead code elimination, inlining

27.5 Debug Info

```

./sage -g program.sage    # Include debug information

```

Chapter 28

Bare-Metal and OS Development

Sage includes a complete pipeline for building bare-metal kernels that boot and run on QEMU for x86_64, aarch64, and RISC-V 64. The boot libraries generate assembly, linker scripts, and QEMU commands from pure Sage code.

28.1 Supported Architectures

Architecture	Boot Method	Serial UART	QEMU Machine
x86_64	Multiboot1 (ELF32)	COM1 port I/O (0x3F8)	default PC
aarch64	Direct ELF load	PL011 MMIO (0x09000000)	virt + cortex-a57
riscv64	Direct ELF load	NS16550 MMIO (0x10000000)	virt (-bios none)

28.2 Quick Start: Build a Kernel with Sage

The `os.boot.build` module generates all files needed for a bootable kernel. Here is a complete example that builds and runs on all three architectures:

```
# build_kernel.sage - Generate a bootable kernel for any architecture
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Pick your architecture: "x86_64", "aarch64", or "riscv64"
let arch = "x86_64"

# Generate boot assembly with serial output
let boot_asm = start.generate_boot_asm_mb1("Hello from SageOS!")
```

```

# Generate linker script
let linker = build.generate_linker_x86_mb1()

# Write files
io.writefile("boot.S", boot_asm)
io.writefile("linker.ld", linker)

# Print build commands
print "Build:"
print "  as --32 -o boot.o boot.S"
print "  ld -m elf_i386 -T linker.ld -o kernel.elf boot.o"
print "Run:"
print "  " + build.qemu_command("x86_64", "kernel.elf")

```

28.3 Example: x86_64 Kernel

Generate boot assembly, compile, and run:

```

gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate self-contained 32-bit multiboot1 kernel with serial
let asm = start.generate_boot_asm_mb1("Hello from SageOS on x86_64!")
let ld = build.generate_linker_x86_mb1()

io.writefile("boot.S", asm)
io.writefile("linker.ld", ld)

```

Build and run:

```

# Assemble (32-bit ELF for multiboot1 compatibility)
as --32 -o boot.o boot.S

# Link
ld -m elf_i386 -T linker.ld -o kernel.elf boot.o

# Run in QEMU
qemu-system-x86_64 -m 128M -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on x86_64!

```

The generated boot stub initializes COM1 at 115200 baud (8N1), prints the message over serial, and halts with cli; hlt.

28.4 Example: aarch64 Kernel

```

gc_disable()
import io

```

```

import os.boot.start as start
import os.boot.build as build

# Generate aarch64 boot assembly + serial (PL011 at 0x09000000)
let asm = start.emit_start_aarch64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_aarch64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("aarch64", "Hello from SageOS on aarch64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)

Build and run:

# Assemble
aarch64-linux-gnu-as -o boot.o boot.S

# Compile kernel C
aarch64-linux-gnu-gcc -ffreestanding -nostdlib -c -o kernel.o kernel.c

# Link (base address 0x40000000 for QEMU virt)
aarch64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU
qemu-system-aarch64 -machine virt -cpu cortex-a57 -m 128M \
    -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on aarch64!

```

The boot stub disables interrupts (DAIF), sets up the stack, zeroes BSS, then jumps to `kmain`. The serial driver uses the PL011 UART with FIFO enabled.

28.5 Example: RISC-V 64 Kernel

```

gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate riscv64 boot assembly + serial (NS16550 at 0x10000000)
let asm = start.emit_start_riscv64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_riscv64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("riscv64", "Hello from SageOS on riscv64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)

```

Build and run:

```
# Assemble
riscv64-linux-gnu-as -march=rv64gc -mabi=lp64d -o boot.o boot.S

# Compile kernel C
riscv64-linux-gnu-gcc -ffreestanding -nostdlib -march=rv64gc \
    -mabi=lp64d -c -o kernel.o kernel.c

# Link (base address 0x80000000 for QEMU virt)
riscv64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU (must use -bios none to skip OpenSBI)
qemu-system-riscv64 -machine virt -m 128M \
    -display none -serial mon:stdio -bios none -kernel kernel.elf
# Output: Hello from SageOS on riscv64!
```

The boot stub disables machine-mode interrupts (mstatus.MIE), sets up the stack, zeroes BSS, and calls `kmain`. The serial driver uses a 16550-compatible UART in MMIO mode.

28.6 Boot Library Reference

Module	Purpose
<code>os.boot.start</code>	Boot assembly generation (multiboot, long mode, BSS init)
<code>os.boot.build</code>	Build pipeline (serial drivers, kernel templates, QEMU commands)
<code>os.boot.linker</code>	Linker script generation (x86_64, aarch64, riscv64)
<code>os.boot.multiboot</code>	Multiboot2 header construction and parsing
<code>os.boot.gdt</code>	GDT descriptor tables (x86_64 long mode)
<code>os.serial</code>	UART configuration and assembly emission (3 architectures)
<code>os.qemu</code>	QEMU VM configuration and command generation

28.7 Kernel Libraries

Module	Purpose
<code>os.kernel.kmain</code>	Kernel initialization pipeline (6 phases)
<code>os.kernel.console</code>	VGA text mode + framebuffer console
<code>os.kernel.pmm</code>	Physical memory manager (bitmap allocator)
<code>os.kernel.vmm</code>	Virtual memory manager (4-level paging)
<code>os.idt</code>	Interrupt descriptor tables (x86/aarch64/riscv64)
<code>os.uefi</code>	UEFI memory map and ACPI table parsing

Module	Purpose
os.acpi	MADT, FADT, HPET, MCFG parsers

28.8 Debugging with GDB

All architectures support GDB debugging via QEMU's `-s -S` flags:

```
# Start QEMU paused, waiting for GDB on port 1234
qemu-system-x86_64 -m 128M -display none -serial mon:stdio \
    -kernel kernel.elf -s -S &

# Connect GDB
gdb kernel.elf -ex 'target remote :1234' -ex 'break _start' -ex 'continue'
```

For cross-architecture debugging, use the appropriate GDB:

```
# aarch64
aarch64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'

# riscv64
riscv64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'
```

Chapter 29

Part VI: Tooling

Chapter 30

Developer Tools

30.1 REPL

Launch the interactive read-eval-print loop:

```
./sage
# or
./sage --repl

sage> 2 + 3
5
sage> let x = "hello"
sage> print upper(x)
HELLO
sage> proc square(n): return n * n end
sage> print square(7)
49
sage> :quit
```

The REPL supports multiline input, error recovery, and all language features.

30.2 Formatter

Automatically format Sage source files:

```
./sage --fmt program.sage
# or
./sage fmt program.sage
```

The formatter normalizes indentation, spacing, and code layout. Both C and self-hosted (Sage) implementations are available.

30.3 Linter

Check code for style issues and common mistakes:

```
./sage --lint program.sage  
# or  
./sage lint program.sage
```

The linter reports unused variables, naming convention violations, overly complex functions, and other code quality issues.

30.4 Type Checker

Run static type analysis:

```
./sage check program.sage
```

The type checker performs best-effort type inference and reports type mismatches, unreachable code, and annotation violations.

30.5 Safety Analyzer

Run the compile-time safety analysis:

```
./sage safety program.sage  
./sage --strict-safety program.sage
```

Reports ownership violations, borrow conflicts, lifetime errors, and nil-safety issues.

30.6 Language Server (LSP)

Start the LSP server for editor integration:

```
./sage --lsp  
# or  
./sage-lsp
```

The LSP server provides:

- **Autocompletion** of functions, variables, and module members
- **Diagnostics** (errors and warnings as you type)
- **Hover info** (type information and documentation)
- **Go-to-definition** navigation

Both C and self-hosted implementations exist. Compatible with VS Code, Neovim, and any editor supporting the Language Server Protocol.

Chapter 31

Build System

31.1 Makefile Targets

```
make                # Build the sage interpreter
make test           # Run interpreter tests (269+ tests)
make test-selfhost  # Run self-hosted tests (1567+ tests)
make test-all      # Run all tests

# Bare-metal / QEMU
make qemu-bare      # Run kernel in QEMU (x86_64)
make qemu-bare-arm64 # Run kernel in QEMU (aarch64)
make qemu-debug     # Debug kernel with GDB
make kernel-bare     # Compile bare-metal kernel
make kernel-uefi     # Compile UEFI application
```

31.2 Version

The version is stored in a single **VERSION** file at the repository root. All build systems (Makefile, CMakeLists.txt, build.sh, sagemake) read from this file automatically. Current version: **3.1.3**.

Chapter 32

Part VIb: Metaprogramming

Chapter 33

Compile-Time Execution

Sage supports compile-time code execution through the `comptime` keyword. Because Sage has a built-in interpreter, any Sage code can run during compilation to produce constants, lookup tables, or configuration data that is baked directly into the binary.

33.1 Comptime Blocks

A `comptime:` block executes its body at compile time. Variables defined inside a `comptime` block are available to subsequent code.

```
# Pre-compute a lookup table
comptime:
  let SINE_TABLE = []
  for i in range(256):
    push(SINE_TABLE, math.sin(i * math.pi / 128.0))
  end
end

proc get_sine(angle):
  return SINE_TABLE[angle % 256]
end
```

33.2 Comptime Expressions

The `comptime(expr)` form evaluates a single expression at compile time:

```
proc factorial(n):
  if n <= 1:
    return 1
  return n * factorial(n - 1)

# Computed at compile time, baked as a constant
let FACT_10 = comptime(factorial(10))
```

Chapter 34

Pragmas and Decorators

Pragmas attach metadata to declarations using the `@` symbol. They inform the compiler backends how to handle specific functions, structs, or blocks.

34.1 Syntax

```
@pragma_name  
@pragma_name("argument")
```

34.2 Built-in Pragmas

Pragma	Target	Effect
<code>@inline</code>	<code>proc</code>	Hints the C backend to emit <code>static inline</code>
<code>@packed</code>	<code>struct</code>	Emits <code>#pragma pack(push, 1)</code> for no padding
<code>@section("name")</code>	<code>proc</code>	Places function in a specific ELF section
<code>@align("N")</code>	<code>struct</code>	Alignment hint for struct layout
<code>@deprecated</code>	<code>proc</code>	Marks a function as deprecated
<code>@noreturn</code>	<code>proc</code>	Marks a function that never returns

34.3 Examples

```
@packed  
struct Ipv4Header:  
    version_ihl: u8  
    tos: u8
```

```
    total_length: u16

@inline
proc fast_math(x, y):
    return x * y

@section(".multiboot_header")
proc boot_header():
    pass
```

34.4 Multiple Pragmas

Multiple pragmas can be stacked on a single declaration:

```
@inline
@deprecated
proc old_multiply(a, b):
    return a * b
```

Chapter 35

AST Macros

Macros are functions that execute at compile time and operate on code structure. In the current implementation, macros are defined with the `macro` keyword and behave as compile-time procedures.

35.1 Macro Definition

```
macro log_call(label):  
  print "entering " + label  
  # ... macro body ...  
  print "exiting " + label
```

```
log_call("main")
```

In interpreter mode, macros are treated as regular functions. In compiled mode, macro bodies are expanded at the call site during compilation.

35.2 Future: Quote and Unquote

The `quote` and `unquote` keywords are reserved for future AST macro support, enabling macros that manipulate Abstract Syntax Tree nodes directly:

```
macro safe_zone(body):  
  return quote:  
    sys.enable_strict_safety()  
    defer sys.restore_safety()  
    unquote(body)  
end
```

Chapter 36

Generics

Sage supports generic type parameters on procedures and structs using bracket syntax `[T, U, ...]`. In the current implementation, Sage uses dynamic typing, so generic parameters serve as documentation and enable future monomorphization in compiled backends.

36.1 Generic Procedures

```
proc identity[T](x: T) -> T:  
    return x
```

```
proc swap[T](a: T, b: T):  
    let tmp = a  
    a = b  
    b = tmp
```

36.2 Generic Structs

```
struct Pair[A, B]:  
    first: A  
    second: B
```

```
struct Stack[T]:  
    items: Array[T]  
    size: Int
```

36.3 Monomorphization (Planned)

When compiled with the C or LLVM backend, the compiler will generate specialized versions for each concrete type used:

```
# At compile time, generates Stack_Int and Stack_String  
let int_stack = Stack[Int]()  
let str_stack = Stack[String]()
```

Chapter 37

Part VIc: Garbage Collection

37.1 GC Modes

Sage provides three garbage collection strategies, selectable at startup:

```
sage --gc:tracing file.sage    # Default: concurrent mark-sweep
sage --gc:arc file.sage       # Automatic Reference Counting
sage --gc:orc file.sage       # Optimized RC with cycle detection
```

37.1.1 Tracing GC (Default)

Concurrent tri-color mark-sweep with SATB write barriers:

- Sub-millisecond stop-the-world pauses (root scan ~50-200us)
- Adaptive threshold triggering based on allocation pressure
- New objects born BLACK (allocated-black invariant)
- Sweeping in 256-object batches to bound pause times

37.1.2 ARC (Automatic Reference Counting)

Deterministic memory reclamation via reference counts:

- Objects freed immediately when reference count reaches zero
- Predictable memory usage (no deferred collection)
- Convenience macros: `ARC_RETAIN`, `ARC_RELEASE`, `ARC_ASSIGN`
- Cannot collect reference cycles — use ORC for cyclic structures

37.1.3 ORC (Optimized Reference Counting)

Nim-inspired ORC combining ARC with Lins' trial deletion cycle collector:

- Three-phase cycle detection: mark PURPLE candidates, trial-decrement scan, collect WHITE garbage
- Triggers cycle collection every 500 decrements (more aggressive than ARC's 1000)
- Recommended for programs with complex object graphs (linked lists, trees, circular references)

37.2 GC API

```
gc_collect()      # Force a collection cycle
gc_stats()        # Print GC statistics
gc_enable()       # Enable automatic collection
gc_disable()      # Disable automatic collection (manual only)
gc_set_arc()       # Switch to ARC mode at runtime
gc_set_orc()       # Switch to ORC mode at runtime
gc_mode()         # Returns "tracing", "arc", or "orc"
```

Chapter 38

Part VIId: Kotlin/Android Backend

Sage can transpile to Kotlin and generate complete Android projects from a single `.sage` file.

38.1 Emitting Kotlin

```
sage --emit-kotlin app.sage -o app.kt           # Transpile to Kotlin
sage --emit-kotlin app.sage -o app.kt -O2      # With type specialization
```

The transpiler maps all Sage constructs to idiomatic Kotlin:

Sage	Kotlin
<code>let x = 10</code>	<code>var x = S.num(10.0)</code>
<code>proc foo(a):</code>	<code>fun foo(a: SageVal): SageVal</code>
<code>class Dog(Animal):</code>	<code>open class Dog : Animal()</code>
<code>for x in items:</code>	<code>for (x in S.toIterable(items))</code>
<code>match x:</code>	<code>when-chain with S.equal()</code>
<code>try: ... catch e:</code>	<code>try { } catch (_e: SageException)</code>
<code>yield val</code>	<code>yield(val) inside sequence { }</code>
<code>await expr</code>	<code>kotlinx.coroutines.runBlocking { expr }</code>
<code>super.init(x)</code>	<code>super.sageInit(x)</code>

38.2 Building Android Apps

```
sage --compile-android app.sage -o my_app \
    --package com.example.app \
    --app-name "My App" \
    --min-sdk 24
```

This generates a complete Gradle project:

```
my_app/
  build.gradle.kts
  settings.gradle.kts
  app/
```

```

build.gradle.kts
src/main/
    AndroidManifest.xml
    kotlin/<package>/Main.kt           # Transpiled Sage
    kotlin/<package>/MainActivity.kt    # Android launcher
    kotlin/sage/runtime/SageRuntime.kt
    res/values/strings.xml, styles.xml

```

Build with: `cd my_app && ./gradlew assembleDebug`

38.3 Type Specialization (-O2+)

At optimization level 2+, the transpiler emits native Kotlin types for variables initialized with literals:

```

let x = 10           # emits: var x: Double = 10.0
let name = "Sage"    # emits: var name: String = "Sage"
let flag = true      # emits: var flag: Boolean = true

```

This eliminates SageVal boxing overhead on hot paths.

38.4 Generators

Generator functions transpile to Kotlin `sequence { }` blocks:

```

proc count_up(n):
    let i = 0
    while i < n:
        yield i
        i = i + 1

```

Emits:

```

fun count_up(n: SageVal): Sequence<SageVal> = sequence {
    var i = S.num(0.0)
    while (S.truthy(S.lt(i, n))) {
        yield(i)
        i = S.add(i, S.num(1.0))
    }
}

```

38.5 Async/Await (Coroutines)

Async procs emit `suspend fun`; await uses `kotlinx.coroutines.runBlocking`:

```

async proc fetch():
    return 42

```

```

let result = await fetch()

```

38.6 Memory Operations on Android

`mem_alloc/mem_read/mem_write/mem_free` map to `java.nio.ByteBuffer`:

```
let buf = mem_alloc(256)
mem_write(buf, 0, "int", 42)
print(mem_read(buf, 0, "int"))    # 42
mem_free(buf)
```

38.7 Jetpack Compose

When `import android.compose` is detected, the project generator uses Compose:

- `@Composable` Activity with Material 3
- Compose BOM, navigation-compose, ui-tooling dependencies
- `ComponentActivity + setContent { }` instead of programmatic views

38.8 GPU Graphics on Android

The `lib/android/graphics.sage` module provides Vulkan-style and OpenGL ES APIs:

```
import android.graphics

let gpu = GPUContext("My App")
gpu.initialize()

# Create compute buffer and run shader
let buf = gpu.create_buffer(1024, "storage")
gpu.upload(buf, [1.0, 2.0, 3.0])
gpu.dispatch_compute("shader.comp", buf, 256)
let result = gpu.download(buf)

# OpenGL ES convenience
let gl = GLESContext()
gl.initialize()
gl.clear(0.1, 0.1, 0.2, 1.0)
gl.draw_arrays("triangles", 0, 3)
gl.swap_buffers()
```

38.9 Concurrency on Android

Kotlin transpiler maps Sage concurrency to JVM primitives:

```
# Atomics → java.util.concurrent.atomic.AtomicLong
let counter = atomic_new(0)
atomic_add(counter, 1)

# Semaphores → java.util.concurrent.Semaphore
let sem = sem_new(3)
```

```
sem_wait(sem)
sem_post(sem)
```

```
# CPU info (via Runtime.getRuntime())
let cores = cpu_count()
```

38.10 Additional Mapped Builtins

The Kotlin transpiler now maps 60+ built-in functions: - **Strings**: upper, lower, strip, split, join, replace, chr, ord - **Paths**: path_join, path_exists, path_basename, path_dirname, path_ext - **Timing**: clock() (System.nanoTime) - **Hashing**: hash(v) (Object.hashCode)

Chapter 39

Part VIe: Performance Optimization

39.1 Hybrid JIT/AOT Architecture

The self-hosted interpreter implements a profile-guided type specialization engine that runs automatically (no flags needed):

39.1.1 How It Works

1. **Profiling Phase:** Every function call records the argument types and increments a call counter. After `HOT_THRESHOLD` (50) calls, the function is analyzed.
2. **Monomorphic Detection:** If all observed calls passed the same argument types (e.g., always numbers), the function is marked as “specialized”.
3. **Type-Feedback Interpretation:** Specialized functions always hit the number fast-path in `eval_binary` (inline arithmetic, no dispatch table lookup). This gives 100% fast-path rate vs ~70% without profiling.
4. **Loop Specialization:** While-loops profile the first 8 iterations. If the body never returns break/return/continue signals, the loop switches to a “fast mode” that skips signal checking entirely — a ~30% speedup on tight loops.

39.1.2 Example

```
proc fib(n):  
  if n <= 1:  
    return n  
  return fib(n - 1) + fib(n - 2)
```

```
# After ~50 calls, fib() is profiled as monomorphic (all-number args).  
# The number fast-path in eval_binary is taken on every binary operation.  
print fib(28)
```

39.1.3 Profiling API

```
# These are internal but accessible:
let profile = _get_profile("fib") # Get profile for a function
# profile["calls"] → 317811 (call count)
# profile["arg_types"] → ["number"]
# profile["monomorphic"] → true
# profile["specialized"] → true
```

39.2 Performance Library (lib/perf.sage)

The `perf` module provides reusable optimization primitives:

39.2.1 Signal Singletons

Eliminate dict allocation on every statement return:

```
import perf

# Instead of allocating {"kind": 0, "value": nil} every time:
let result = perf.sig_normal_nil() # Returns pre-allocated singleton
let brk = perf.sig_break() # Zero allocation
```

39.2.2 Dispatch Tables

Replace if/elif chains with O(1) dict lookup:

```
import perf

let table = perf.make_dispatch_table()
perf.dispatch_register(table, "add", proc(args): return args[0] + args[1])
perf.dispatch_register(table, "sub", proc(args): return args[0] - args[1])

# O(1) dispatch instead of N comparisons:
let result = perf.dispatch_call(table, "add", [10, 20])
```

39.2.3 Shape Objects

Pre-shaped dict constructors for known structures:

```
import perf

let func = perf.shape_function("add", params, body, closure, false)
let cls = perf.shape_class("Dog", parent)
let inst = perf.shape_instance(cls)
let env = perf.shape_env(parent_env)
```

39.2.4 Fast Numeric Operations

Bypass type dispatch for known-numeric paths:

```
import perf

let sum = perf.fast_sum([1, 2, 3, 4, 5])
let total = perf.fast_add_num(a, b)
```

39.2.5 Flat Environment Cache

O(1) variable access bypassing scope chains in tight loops:

```
import perf

let cache = perf.flat_cache_snapshot(env)
# ... hot loop using flat_cache_get/set ...
perf.flat_cache_flush(cache, env)
```

39.3 Self-Hosted Interpreter Optimizations

The self-hosted interpreter (`src/sage/interpreter.sage`) applies these patterns:

1. **Pre-allocated signal singletons:** `result_normal(nil)`, `result_break()`, `result_continue()` return cached dicts
2. **Native dispatch table:** `call_native()` uses O(1) dict lookup instead of 180-line if/elif chain
3. **Binary op dispatch table:** `eval_binary()` uses O(1) dict lookup for all operators
4. **Recursion depth at call boundaries only:** `eval_expr()` no longer increments/decrements depth counter
5. **Shape constructors:** functions, classes, environments built as single dict literals

39.4 Native C Interpreter Optimizations

The C interpreter (`src/c/interpreter.c`, `src/c/env.c`) applies:

1. **Cached name length in EnvNode:** `name_length` field avoids `strlen` during lookup
2. **memcmp with length pre-check:** `env_get/env_define/env_assign` check `name_length == length` before `memcmp`
3. **Inlined eval_expr:** recursion depth checked only at `interpret()` boundaries, not per-expression
4. **For-loop slot caching:** loop variable node pointer cached after first `env_define`, subsequent iterations write directly
5. **String pointer equality:** `values_equal()` checks `AS_STRING(a) == AS_STRING(b)` before `strcmp`

39.5 Cross-Backend Benchmarks

Run the 8-workload benchmark across all backends:

```
bash benchmarks/run_backend_compare.sh
python3 scripts/generate_backend_chart.py
```

Workloads: fibonacci, loop sum, array ops, string concat, dict ops, prime sieve, nested loops, LCG hash.

Chapter 40

Part VI: Default Runtime and Execution Modes

40.1 JIT+AOT Hybrid Default

As of v3.3.0, Sage’s default runtime is `auto` — JIT profiling mode on hosted platforms, AST interpreter on bare-metal:

Environment	Auto Resolves To	Why
Desktop/Server	JIT profiling	Full OS, type feedback collection
Android	JIT profiling	JVM + coroutines available
Bare-metal/Pico	AST interpreter	No fork/system, safe fallback
Kernel dev	SageMetal VM	Freestanding, zero allocation

Override with `--runtime ast`, `--runtime bytecode`, `--runtime jit`, or `--runtime aot`.

40.2 Runtime Pragma Decorators

Control JIT/AOT behavior per-function:

```
@nojit
proc low_level_driver():
    # Skips JIT profiling for this function
    mem_write(ptr, 0, "int", 42)
```

```
@noaot
proc dynamic_dispatch():
    # Skips AOT compilation
    ...
```

```
@noprofile
proc realtime_handler():
```

```
# Skips all profiling overhead  
...
```

Chapter 41

Part VIg: SageMetal VM — Bare-Metal Bytecode Interpreter

The SageMetal VM is a freestanding bytecode interpreter that runs without an OS, libc, or dynamic memory allocation. It is designed for kernels, bootloaders, embedded systems, and OS development.

41.1 Architecture

All storage is static — no malloc, no heap fragmentation:

- **Value Stack:** 512 slots (configurable via `METAL_STACK_SIZE`)
- **Constant Pool:** 1024 entries
- **String Pool:** 32KB bump allocator (interned, deduplicated)
- **Heap:** 64KB bump allocator for general use
- **Scope Chain:** 64 levels deep, 32 variables per scope
- **Array Pool:** Fixed-capacity arrays (256 elements each)

41.2 MetalValue Type

Compact 16-byte tagged union:

<code>MV_NIL</code>	- null value
<code>MV_NUM</code>	- IEEE 754 double
<code>MV_BOOL</code>	- 0 or 1
<code>MV_STR</code>	- index into string pool
<code>MV_ARR</code>	- index into array pool
<code>MV_DICT</code>	- index into dict pool
<code>MV_FN</code>	- index into function table
<code>MV_PTR</code>	- raw pointer (for MMIO, DMA)

41.3 Host I/O Callbacks

The kernel or bootloader sets these before running the VM:

```
MetalVM vm;
metal_vm_init(&vm);
vm.write_char = serial_putchar;    // Console output
vm.read_char = serial_getchar;    // Console input
vm.write_port = x86_outb;         // Port I/O
vm.read_port = x86_inb;          // Port I/O
vm.map_mmio = identity_map;      // MMIO mapping

metal_vm_load(&vm, bytecode, length);
metal_vm_run(&vm);    // Execute until halt
```

41.4 Single-Step Mode

For cooperative multitasking in kernels:

```
while (metal_vm_step(&vm)) {
    // Check interrupts, handle timer ticks, etc.
    if (timer_interrupt_pending) handle_timer();
}
```

41.5 Building

```
make metal-vm    # Produces obj/metal_vm.o (freestanding)
```

Link with your kernel:

```
gcc -ffreestanding -nostdlib -o kernel.elf \
    boot.o kernel.o obj/metal_vm.o -lgcc
```

Chapter 42

Part VIh: Metal Standard Library

The `lib/metal/` modules provide bare-metal primitives for kernel and embedded development.

42.1 metal.core

Core primitives: serial I/O, port I/O, MMIO, CPU control, memory.

```
import metal.core

# Console
core.puts("Hello from kernel!")

# Port I/O (x86)
core.outb(0x3F8, 65)    # Send 'A' to COM1
let val = core.inb(0x60) # Read keyboard scancode

# MMIO
core.mmio_write32(0xB8000, 0x0F41) # Write 'A' to VGA

# CPU control
core.cli()    # Disable interrupts
core.sti()    # Enable interrupts
core.hlt()    # Halt until next interrupt

# Bump allocator
core.heap_init(0x100000, 65536)
let ptr = core.heap_alloc(256)
```

42.2 metal.serial

UART drivers for x86 (NS16550A) and ARM (PL011):

```
import metal.serial
```

```
# x86 COM1
serial.uart_init(serial.COM1, 115200)
serial.uart_puts(serial.COM1, "Boot complete\n")

# ARM PL011
serial.pl011_init(0x09000000)
serial.pl011_puts(0x09000000, "Hello from ARM\n")
```

42.3 metal.irq

Interrupt management and PIC control:

```
import metal.irq

# Remap PIC to vectors 32-47
irq.pic_remap(32, 40)

# Register handler for timer interrupt
irq.register_handler(32, proc(vector):
    # Handle timer tick
    irq.pic_eoi(irq.IRQ_TIMER)
)

# Enable timer IRQ
irq.pic_unmask(irq.IRQ_TIMER)
```

42.4 metal.timer

Hardware timer with tick counting and sleep:

```
import metal.timer

timer.pit_init(1000)    # 1000 Hz (1ms resolution)

let start = timer.ticks()
# ... do work ...
let elapsed = timer.stopwatch_elapsed_ms(start)

timer.sleep_ms(500)     # Sleep 500ms
```

42.5 metal.gpio

GPIO pin control for embedded targets:

```
import metal.gpio

gpio.gpio_init(0x40000000, 32) # Init 32 pins at MMIO base
```

```
gpio.pin_mode(25, gpio.PIN_OUTPUT)  # LED pin
gpio.led_blink(25, 5, 250)           # Blink 5 times, 250ms interval
```

Chapter 43

Part VII: Appendices

Chapter 44

Appendix A: Built-in Functions

The following functions are available globally without any imports.

44.1 Core Functions

Function	Description
<code>print(value)</code>	Print value to stdout with newline
<code>input(prompt)</code>	Read a line from stdin
<code>len(x)</code>	Length of string, array, dict, or tuple
<code>type(x)</code>	Type name as string
<code>str(x)</code>	Convert any value to string
<code>tonumber(s)</code>	Parse string to number
<code>int(x)</code>	Truncate number to integer
<code>chr(n)</code>	Integer code point to character
<code>ord(c)</code>	Character to integer code point
<code>clock()</code>	CPU time in seconds (high resolution)
<code>hash(x)</code>	Hash value of a string
<code>doc(f)</code>	Get doc comment for a function

44.2 Array Functions

Function	Description
<code>push(arr, val)</code>	Append value to end of array
<code>append(arr, val)</code>	Alias for <code>push</code>
<code>pop(arr)</code>	Remove and return last element
<code>range(n)</code>	Array [0, 1, ..., n-1]
<code>range(a, b)</code>	Array [a, a+1, ..., b-1]
<code>slice(arr, start, end)</code>	Sub-array from start to end
<code>array_extend(a, b)</code>	Extend array a with elements of b

44.3 String Functions

Function	Description
<code>split(str, delim)</code>	Split string by delimiter
<code>join(arr, delim)</code>	Join array elements with delimiter
<code>replace(str, old, new)</code>	Replace all occurrences
<code>upper(str)</code>	Convert to uppercase
<code>lower(str)</code>	Convert to lowercase
<code>strip(str)</code>	Trim leading/trailing whitespace
<code>startswith(str, prefix)</code>	Check if string starts with prefix
<code>endswith(str, suffix)</code>	Check if string ends with suffix
<code>contains(str, sub)</code>	Check if string contains substring
<code>indexof(str, sub)</code>	Find index of substring (-1 if not found)

44.4 Dictionary Functions

Function	Description
<code>dict_keys(d)</code>	Array of all keys
<code>dict_values(d)</code>	Array of all values
<code>dict_has(d, key)</code>	Check if key exists
<code>dict_delete(d, key)</code>	Remove key from dictionary

44.5 Generator Functions

Function	Description
<code>next(gen)</code>	Advance generator and return next value

44.6 GC Functions

Function	Description
<code>gc_collect()</code>	Force a garbage collection cycle
<code>gc_stats()</code>	Return GC statistics as a dict
<code>gc_collections()</code>	Return number of GC cycles performed
<code>gc_enable()</code>	Enable the garbage collector
<code>gc_disable()</code>	Disable the garbage collector
<code>gc_set_arc()</code>	Switch to ARC mode at runtime
<code>gc_set_orc()</code>	Switch to ORC mode at runtime
<code>gc_mode()</code>	Return current GC mode string

44.7 Path Functions

Function	Description
<code>path_join(a, b)</code>	Join two path components
<code>path_dirname(path)</code>	Directory portion of path
<code>path_basename(path)</code>	Filename portion of path
<code>path_ext(path)</code>	File extension
<code>path_exists(path)</code>	Check if path exists
<code>path_is_dir(path)</code>	Check if path is a directory
<code>path_is_file(path)</code>	Check if path is a regular file

44.8 Bytes Functions

Function	Description
<code>bytes(n)</code>	Create byte buffer of size n
<code>bytes_len(b)</code>	Length of byte buffer
<code>bytes_get(b, i)</code>	Get byte at index
<code>bytes_set(b, i, v)</code>	Set byte at index
<code>bytes_to_string(b)</code>	Convert bytes to string
<code>bytes_slice(b, start, end)</code>	Slice byte buffer
<code>bytes_push(b, v)</code>	Append byte to buffer

44.9 Memory Functions

Function	Description
<code>mem_alloc(size)</code>	Allocate raw memory
<code>mem_free(ptr)</code>	Free allocated memory
<code>mem_read(ptr, off)</code>	Read value at pointer offset
<code>mem_write(ptr, off, val)</code>	Write value at pointer offset
<code>mem_size(ptr)</code>	Size of allocation
<code>offsetof(val)</code>	Get memory address of a value
<code>sizeof(type)</code>	Size of a type in bytes
<code>ptr_add(ptr, off)</code>	Pointer arithmetic
<code>ptr_to_int(ptr)</code>	Convert pointer to integer

44.10 FFI Functions

Function	Description
<code>ffi_open(lib_path)</code>	Open shared library
<code>ffi_close(handle)</code>	Close shared library
<code>ffi_sym(handle, name)</code>	Look up symbol

Function	Description
<code>ffi_call(sym, args, ret_type)</code>	Call foreign function

44.11 Struct Interop Functions

Function	Description
<code>struct_def(fields)</code>	Define a C struct layout
<code>struct_new(def)</code>	Allocate a new struct
<code>struct_get(ptr, def, field)</code>	Read struct field
<code>struct_set(ptr, def, field, val)</code>	Write struct field
<code>struct_size(def)</code>	Size of struct in bytes

44.12 Assembly Functions

Function	Description
<code>asm_exec(code)</code>	Execute inline assembly
<code>asm_compile(code)</code>	Compile assembly to machine code
<code>asm_arch()</code>	Get current architecture string

Chapter 45

Appendix B: CLI Reference

Usage: sage [options] [file.sage]

Running:

(no args)	Interactive REPL
file.sage	Run script in interpreter
--repl	Interactive REPL (explicit)

Compilation:

--emit-c FILE -o OUT	Emit C source code
--compile FILE -o OUT	Compile to binary via C backend
--emit-llvm FILE -o OUT	Emit LLVM IR text
--compile-llvm FILE -o OUT	Compile to binary via LLVM (requires clang)
--emit-asm FILE -o OUT	Emit assembly source
--compile-native FILE -o OUT	Compile to native binary
--compile-bare FILE -o OUT	Bare-metal ELF (no libc)
--compile-uefi FILE -o OUT	UEFI PE application
--emit-kotlin FILE -o OUT	Emit Kotlin source code
--compile-android FILE -o DIR	Generate Android Gradle project
--target ARCH	Target architecture: x86-64, aarch64, rv64

Android Options:

--package PKG	Android package name (default: com.sage.app)
--app-name NAME	Display name for Android app
--min-sdk N	Minimum Android SDK version (default: 24)

Runtime:

--runtime MODE	Execution backend: ast, bytecode, jit, aot, auto
--jit FILE	JIT compile with profiling
--aot FILE	AOT compile to native binary

Garbage Collection:

--gc:tracing	Tracing GC (default, concurrent mark-sweep)
--gc:arc	Automatic Reference Counting

--gc:orc Optimized RC with cycle detection

Optimization:

-O0 No optimization
 -O1 Constant folding
 -O2 Constant folding + DCE + type specialization (Kotlin)
 -O3 Aggressive (+ inlining)
 -g Include debug info

Safety:

--safety Check @safe annotated functions
 --strict-safety Enforce safety globally

Tooling:

--fmt FILE Format source code
 --lint FILE Lint source code
 --lsp Start Language Server Protocol server
 fmt FILE Format source code
 lint FILE Lint source code
 check FILE Run type checker
 safety FILE Run safety analyzer

Self-Hosted Compiler:

sage.sage FILE Run file in self-hosted interpreter
 sage.sage --emit-c FILE Emit C via self-hosted compiler
 sage.sage --emit-llvm FILE Emit LLVM IR via self-hosted compiler
 sage.sage --emit-asm FILE Emit assembly via self-hosted compiler
 sage.sage fmt FILE Format via self-hosted formatter
 sage.sage lint FILE Lint via self-hosted linter
 sage.sage check FILE Type check via self-hosted checker

REPL Commands:

:help Show REPL help
 :env Show environment
 :ast <code> Show parsed AST
 :emit-c <code> Show C backend output
 :emit-llvm <code> Show LLVM IR output
 :emit-kotlin <code> Show Kotlin backend output
 :time <expr> Time expression evaluation
 :bench N <expr> Benchmark expression N times
 :runtime MODE Switch runtime (ast, bytecode, jit, aot)

Chapter 46

Appendix C: Language Gotchas

This section documents known behaviors and design decisions. Items marked **FIXED** were resolved in v1.4 or later.

46.1 Still Relevant

1. **0 is truthy** – Only `false` and `nil` are falsy. This is an intentional design decision. Use explicit comparisons: `if x == 0:` not `if not x:`.
2. **`chr(0)` truncates strings** – Null bytes terminate C strings in the interpreter. The runtime uses C-style null-terminated strings, so inserting `chr(0)` into a string will truncate it at that point.
3. **No wildcard imports** – `from module import *` is not supported. Use `import module` then `module.func()`, or import specific names: `from module import func`.
4. **No multiline dict/array literals** – Build complex structures incrementally with assignments:

```
let config = {}
config["host"] = "localhost"
config["port"] = 8080
```
5. **`match` and `init` are reserved keywords** – `match` cannot be used as a variable name. `init` is reserved but works as a property name after `.` and `->`.
6. **`super` requires explicit `self`** – Write `super.init(self, args)` not `super.init(args)`.

46.2 Fixed in v1.4+

7. **~~No escape sequences~~** – **FIXED.** `\n`, `\t`, `\r`, `\\`, `\"`, `\xHH` and other escape sequences now work in string literals.
8. **~~No hex literals~~** – **FIXED.** `0xFF` and `0o755` are now parsed correctly as number literals.
9. **~~`elif` chains malfunction~~** – **FIXED.** Unlimited `elif` branches are now supported without issues.

10. ~~**Instance == always false**~~ – **FIXED**. Structural equality now works for class instances, arrays, and dicts.
11. ~~**push/append mismatch**~~ – **FIXED**. Both `push()` and `append()` work identically in the C interpreter. They are aliases for the same native function.
12. ~~**% casts to int**~~ – **FIXED**. The modulo operator now uses `fmod()` and preserves float semantics. `3.7 % 1` returns `0.7`, not `0`.
13. ~~**Class methods can't see module-level vars**~~ – **FIXED**. Methods now have access to their defining environment via `defining_env`.

Chapter 47

Appendix D: Reserved Keywords

Sage reserves the following 54 keywords. They cannot be used as variable or function names.

47.1 Control Flow

Keyword	Purpose
<code>if</code>	Conditional branch
<code>elif</code>	Else-if branch
<code>else</code>	Default branch
<code>while</code>	While loop
<code>for</code>	For-in loop
<code>in</code>	For-in iteration
<code>break</code>	Exit loop
<code>continue</code>	Skip to next iteration
<code>match</code>	Pattern matching
<code>case</code>	Match case
<code>default</code>	Match default
<code>end</code>	Block terminator

47.2 Functions and Classes

Keyword	Purpose
<code>proc</code>	Function definition
<code>return</code>	Return from function
<code>class</code>	Class definition
<code>self</code>	Instance reference
<code>init</code>	Constructor method
<code>super</code>	Parent class reference
<code>struct</code>	Struct definition
<code>enum</code>	Enum definition
<code>trait</code>	Trait definition

47.3 Variables

Keyword	Purpose
<code>let</code>	Immutable binding
<code>var</code>	Mutable variable

47.4 Values and Logic

Keyword	Purpose
<code>true</code>	Boolean true
<code>false</code>	Boolean false
<code>nil</code>	Null value
<code>and</code>	Logical AND
<code>or</code>	Logical OR
<code>not</code>	Logical NOT

47.5 Exceptions

Keyword	Purpose
<code>try</code>	Begin exception block
<code>catch</code>	Handle exception
<code>finally</code>	Always-execute block
<code>raise</code>	Throw exception

47.6 Advanced

Keyword	Purpose
<code>defer</code>	Deferred execution
<code>yield</code>	Generator yield
<code>async</code>	Async function
<code>await</code>	Await async result
<code>import</code>	Import module
<code>from</code>	From-import
<code>as</code>	Import alias
<code>unsafe</code>	Unsafe block
<code>print</code>	Print statement

Note: `elif` is recognized by the lexer as a combination of `else` + `if` tokens but is effectively reserved. The `end` keyword is used to terminate all block constructs: `proc`, `if`, `while`, `for`, `match`, `case`, `try`, `catch`, `finally`, `class`, `struct`, `enum`, `trait`, and `unsafe`.